

Chaos-Driven Refactoring

Break the system on purpose. Ship the architecture that survives it.

CDR closes the APM-to-code gap: it injects a real fault, captures the physical collapse, has IBM Bob read and repair the affected repository, then reruns the identical experiment to verify resilience.

A complete chaos -> diagnosis -> code change -> verification loop.

The problem: diagnosis stops before the code

- Chaos engineering can reproduce the failure, but does not repair the business logic.
- APM and logs can localize symptoms, but the fix still depends on a human expert.
- Coding agents can edit code, but normally do not see the physical failure that motivated the change.
- CDR connects these steps and refuses to call a patch successful until the same fault is survived.

Target users: SRE, DevOps and platform teams protecting checkout, payment and other high-value paths.

One application, four observable phases

01 Inject chaos k6 + Toxiproxy	02 Classify telemetry + evidence	03 Let Bob rewrite it real repository diff	04 Prove it survives same experiment again
---	---	---	---

Every phase streams to the control room through Supabase, with a deterministic mock mode for rehearsals and a live mode for evidence.

IBM Bob is part of the runtime repair loop

- The worker shallow-clones the submitted repository at the requested commit.
- IBM Bob Shell runs in agent mode inside that clone and reads the actual source code.
- Bob applies the minimal resilience change; the runner extracts the real git diff.
- The patch, changed files, Bob task ID and Bobcoin cost are stored and shown in the dashboard.
- Live verification fails closed unless the patch source is IBM Bob and the expected service was changed.

Fallbacks exist for mock development, but live mode requires Bob. This makes the evaluation claim auditable rather than cosmetic.

Architecture: control plane + data plane

- Control plane: Next.js dashboard on Vercel, Supabase queue, run history and realtime progress.
- Data plane: local Python worker, Docker Compose lab, k6 load, Toxiproxy fault injection and Bob Shell.
- The runner rebuilds the Bob-patched checkoutservice container before the after measurement.
- The same scenario, load profile and fault are executed before and after the repair.

Vercel hosts the control room. Docker, k6 and Bob remain on the runner machine where the target repository and live lab are available.

Measured live result

24.8%	27.3%	30 s -> none	0.083248
p95 latency improvement	throughput increase	collapse eliminated	Bobcoins consumed

- Run: run-2738beb5 | 200 virtual users | 120 seconds | 800 ms emailservice latency.
- Bob added a 200 ms child context around the optional confirmation-email RPC.
- The order response remained successful; payment and shipping semantics were preserved.

The jury demo: one screen, one story

- Open the dashboard and submit the pinned Online Boutique scenario in live mode.
- Show the timeline: physical collapse -> classification -> Bob task attribution -> verification.
- Open the diagnosis card to show the task ID, Bobcoins and identified source file.
- Open the generated patch to show the real diff, not a generic template.
- Compare before/after charts and end on LIVE EVIDENCE with no collapse after the same fault.

```
Demo command: python -m cdr run --scenario scenarios/checkout-latency-cascade.yaml --mode live --sink all
```

Why CDR is different

- It does not stop at an AI explanation: it produces a repository-level change.
- It does not trust the change blindly: it rebuilds the affected service and reruns the fault.
- It makes IBM Bob measurable through task IDs, cost attribution and a real git diff.
- It turns resilience work from a manual incident investigation into a repeatable engineering loop.

CDR: from physical failure to verified resilience, with IBM Bob in the critical path.

Evidence: docs/LIVE_EVIDENCE.md | Runtime integration: docs/BOB_USAGE.md | Demo: docs/DEMO_SCRIPT.md